

What Claude said was wrong

Data assembly — faithful, with one real regression

The good news: `build_visual_features`, `label_debate_visual`, and the `build_seg_all` frame→audio join / identity merge / aggregation all reproduce our agreed pipeline correctly, and it adds a `video` column (needed for multi-video). The schema lines up with what the analysis expects (`prob_*`, the five pose features, `NON_REDUNDANT`, `final_name`, `top_emotion`, `n_frames`).

But the one thing that's missing is the `seg_speaker` mode fix. In `build_seg_all` the flag is just:

python

```
df_all['speaking_face'] = df_all['has_audio'] & (df_all['final_name'] == df_all['speaker'])
```

using the raw per-frame `speaker`, and then aggregation takes `final_name = 'first'`. That's precisely the unsafe combination we found: the per-frame speaker is contaminated at segment boundaries (the `floor+1/ceil` explode + `drop_duplicates(keep='first')` in `label_debate_visual`), so a segment's boundary frames can carry the neighbour's identity, and `'first'` then picks arbitrarily. We fixed this with the mode in our cell 17; the pipeline dropped it. It needs restoring — insert before the aggregation:

python

```
owner = df_all[df_all['has_audio']].groupby('segment_id')['speaker'].agg(_mode_or_nan)
df_all['seg_speaker'] = df_all['segment_id'].map(owner)
df_all['speaking_face'] = df_all['has_audio'] & (df_all['final_name'] == df_all['seg_speaker'])
```

(`_mode_or_nan` is already defined inside the function.) Per debate the effect is small (~1% of frames, as we saw), but it's a correctness regression across 28 debates and trivial to put back.

Two smaller things: `build_seg_all` keeps `top_emotion` (mode of per-frame labels) rather than the `dom_emotion` (argmax of the averaged probs) we settled on — not a bug, just the less-consistent choice, and the analysis uses `top_emotion` so it runs.

And `_cluster_and_label` will `KeyError` on `global_centroids[cA]` if a candidate

appears in fewer than 2 debates; with all 28 debates each candidate appears 7×, so it's safe for the full run but would crash on a subset — worth a guard.

The analysis — this is what needs changing

The B/C/D cells are the **old pooled version**, ported before we worked out the within-candidate control — and pooling is *more* dangerous here than in your single video, because `seg_cands` now mixes all 8 candidates of both sexes and very different voices and emotion profiles:

- **B1 (cell 50)** pooled audio↔emotion across 8 candidates is the maximally confounded view — any strong cell is mostly telling the candidates apart, not measuring emotion-driven voice change. **B5 (cell 58)** pose↔audio and **B3/B4 (54, 56)** audio-by-emotion have the same problem, and **C3 (65)** marks significance on the pooled table, so its stars are largely identity-driven.
- **D (cell 67)** runs CCA on raw pooled features — same issue as the single-video cell you're deleting; it will overstate shared information because both modalities encode identity.

The fix is the one we already validated: lead with the within-candidate versions. And the good news is the tools are **already in the notebook and correct** — **C1 (cell 61)** `within_speaker_corr(group='final_name')` is exactly the right analysis, and **C2 (cell 63)** variance partition (candidate vs emotion η^2) is the perfect quantification of the confound (it'll almost certainly show candidate $\gg$ emotion). So the change is mostly reframing: present B1/B5 as the *naive, confounded baseline* (or as the pooled-vs-within pair we used), draw the real conclusions from C1/C2, and center within candidate before the CCA in D.

One important boundary, so you don't over-correct: the within-candidate control only applies to the *within-person questions* (does emotion change a person's voice/pose). *The poll correlation (cell 74), the per-candidate acoustic profiles (I.B), and voice consistency (I.C) are legitimately between-candidate* — "do better-polling candidates speak differently" *is* a between-person question, so those are fine as-is and should not be centered.

The Fix Claude suggested

In `build_seg_all`, find this block (right after the audio merge):

python

```
df_all = df_all.merge(audio_by_frame, on='frame_number',
how='left')
df_all['has_audio'] = df_all['has_audio'].fillna(False)
df_all['segment_id'] =
```

```
df_all['segment_id'].fillna(-1).astype(int)
df_all['speaking_face'] = (df_all['has_audio'] &
                           (df_all['final_name'] ==
df_all['speaker']))
```

and replace it with:

python

```
df_all = df_all.merge(audio_by_frame, on='frame_number',
how='left')
df_all['has_audio'] = df_all['has_audio'].fillna(False)
df_all['segment_id'] = df_all['segment_id'].fillna(-1).astype(int)

# robust per-segment speaker = the speaker that appears MOST in
the
# segment's frames (the per-frame label is contaminated at
boundaries)
owner = (df_all[df_all['has_audio']]
         .groupby('segment_id')['speaker'].agg(_mode_or_nan))
df_all['seg_speaker'] = df_all['segment_id'].map(owner)
df_all['speaking_face'] = (df_all['has_audio'] &
                           (df_all['final_name'] ==
df_all['seg_speaker']))
```

That's the whole fix. `_mode_or_nan` is already defined inside `build_seg_all` (it's used for `top_emotion`), so nothing new to import, and it's exactly the cell-17 logic from your single-video notebook: group the segment's frames by `segment_id`, take the most frequent `speaker`, and decide `speaking_face` against that instead of the raw per-frame label.

Two optional add-ons while you're in there, neither required:

If you want `seg_speaker` carried onto the segment rows (handy for sanity checks, and harmless), add it to the aggregation dict a few lines down:

python

```
agg['final_name'] = 'first'
agg['seg_speaker'] = 'first' # add this line
```

And if you'd rather use the consistent `dom_emotion` (argmax of the averaged probabilities) instead of the per-frame mode, add this right after the `seg = (...).reset_index()` line:

python

```
seg['dom_emotion'] =  
seg[existing_emotion].idxmax(axis=1).str.replace('prob_', '', regex=False)
```

One practical note: since `seg_all` is cached to `seg_all.pkl`, you'll need to delete that cache (and `labels_all.pkl` isn't affected) so cell 5 rebuilds it with the fix — otherwise it'll just reload the old version. After that, `seg_cands` and everything downstream picks up the corrected `speaking_face`.